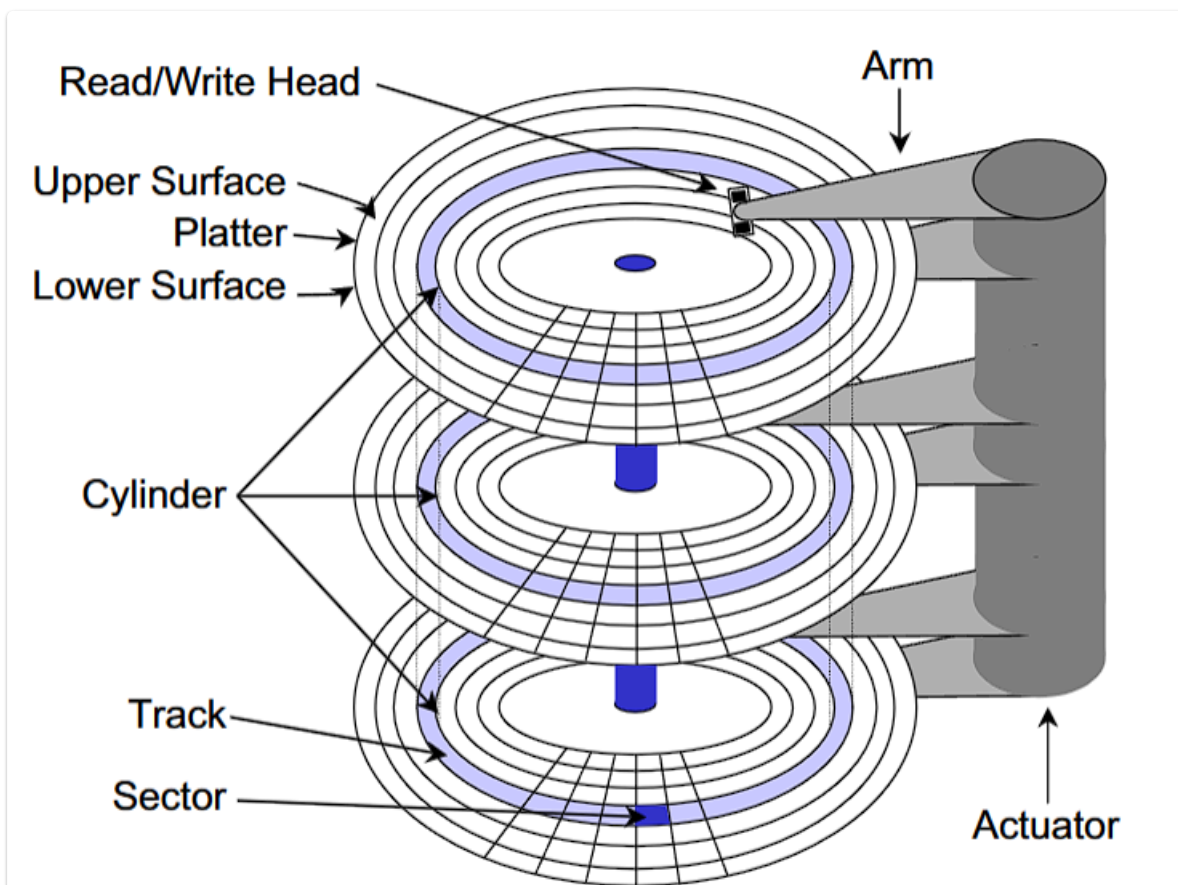


1. Storage

HDD & SSD

Calculating Access Time

- HDD access time = seek time + rotational delay + transfer time
 - (command processing time negligible)
- Seek time
 - Moving arm to move to correct track
 - Avg 5-6 ms
- Rotational delay
 - Time for block to rotate under head
 - Rotation speed in RPM
 - Average rotation = 1/2 rotation
 - 10000 RPM, rotational delay = $0.5 * 60 / 10000 = 3\text{ms}$
- Transfer time
 - n = No. of requested sectors on same track
 - Transfer time = $n * \frac{\text{time for one revolution}}{\text{number of sectors per track}}$



SSD Comparison

- 100x faster random IO
- Slightly faster sequential IO
- Lower power consumption
- Disadvantage: Erasure cycle - before overwrite, need to erase then overwrite; limited number of erasure

Buffer Pool

Buffer manager implementation

- Pin count: For clients to "pin" a page - indicate they are using the page
- Dirty flag: Already updated page

Buffer replacement policy

- Clock replacement strategy
- If pin count = 0, go next
- If ref bit = 1, set ref bit = 0
- Else, replace, set pin count to 1, return address of frame

Heap File Implementations

NOT heap memory, heap file (random order file)

Linked List

- 2 doubly linked lists for available (including partially) and full pages

Disadvantage

- Worst case $O(n)$ to find available page

Page Directory

- Use a page (directory) to store information about a chunk of pages
- LinkedList all directories

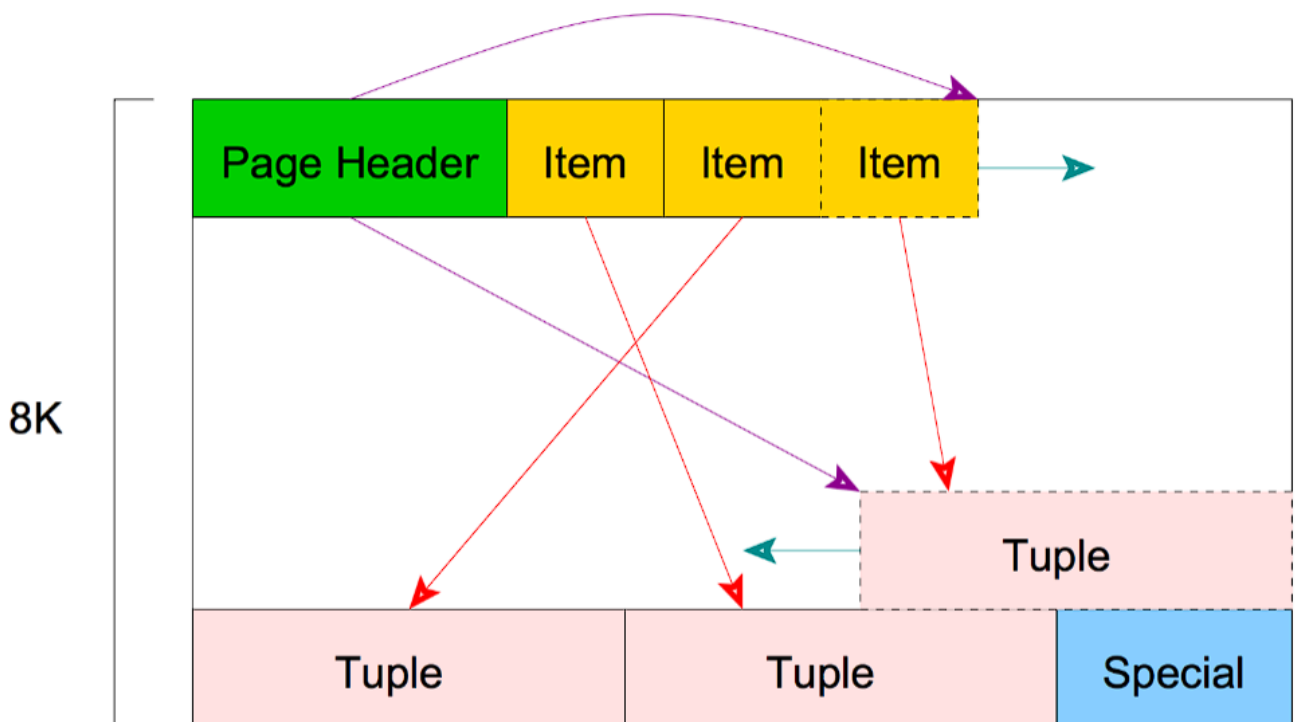
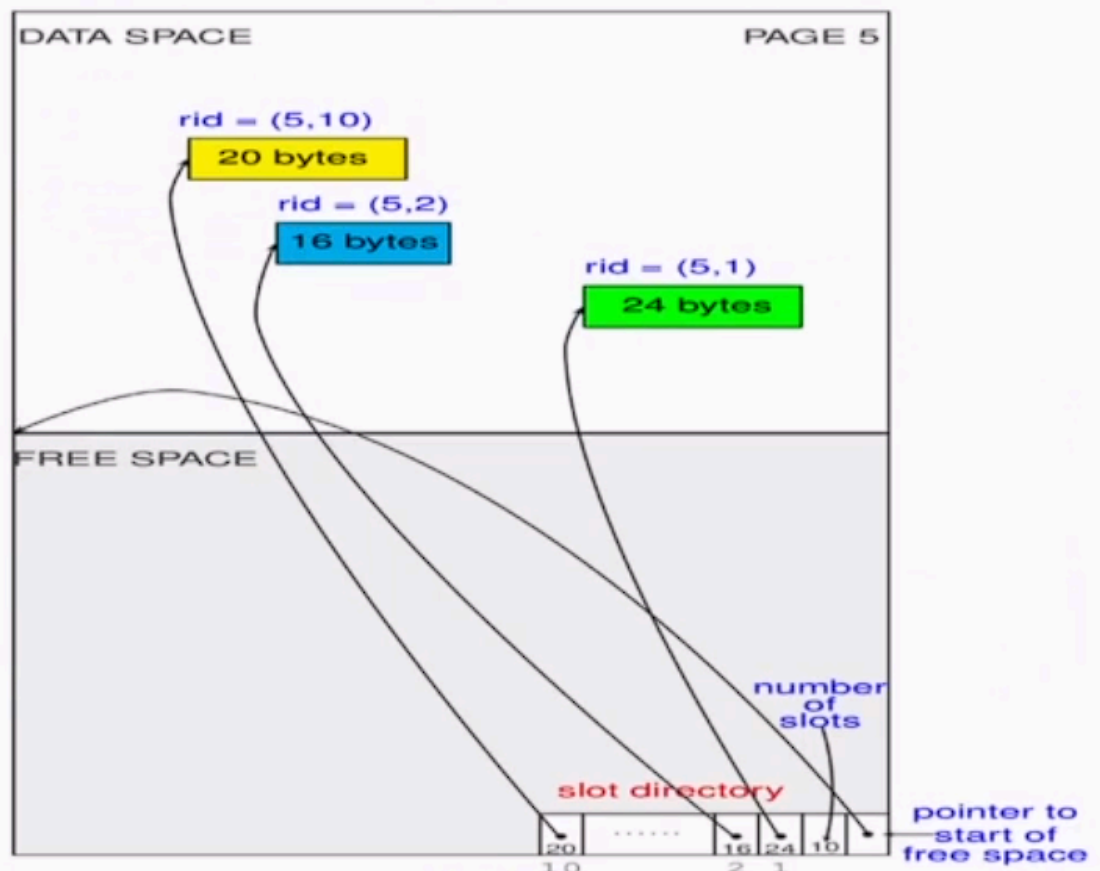
Advantage

- Only need to scan the directory pages and 1 data page in worst case

Page Format

- How records organized in page
- Record ID (RID) = {page_id, slot_number}
- Fixed-length Records
 - Packed organization: Store records in contiguous slots (Store a counter for number of records)
 - Difficult to pack and need change RID on every update
 - Unpacked organization: Use bit array to maintain free slots
 - No need to update RID, just update bitmap
- Variable-length record
 - At end of free space, keep dynamic data structure to point to each record
 - Use a record at the end to point to your next free space

Variable-Length Records: Slotted Page Organization



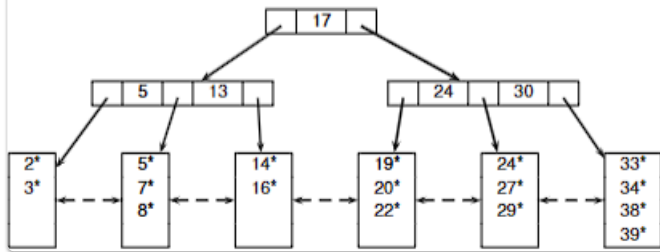
2. Tree-Index

B^+ -Tree Structure Basics

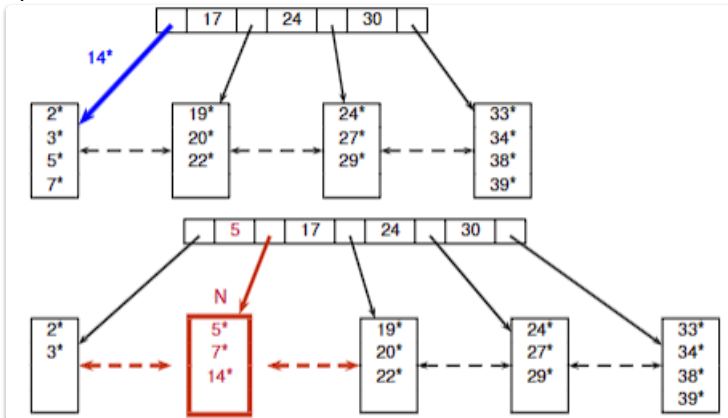
- Similar to CS2040s
- Internal nodes store index entries in the form $p_0, k_1, p_1, k_2, p_2, \dots, p_n$
 - Where values in page p_1 have a key $\geq k_1$

- Leaves are doubly-linked
- Order of index tree, d
 - Each non-root node contains m entries where $m \in [d, 2d]$
 - Root node contains m entries where $m \in [1, 2d]$
 - Note: entry = keys

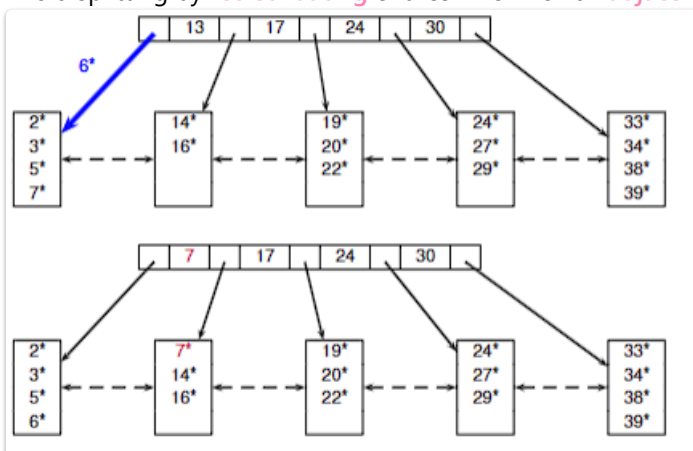
Example: B⁺-tree with order = 2



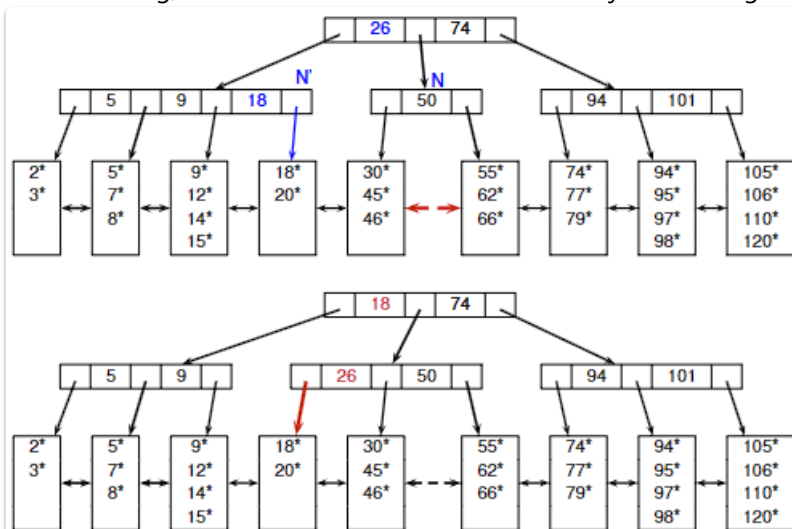
- Split when a node has $> 2d$ entries, with $d + 1$ in new leaf node (on right)



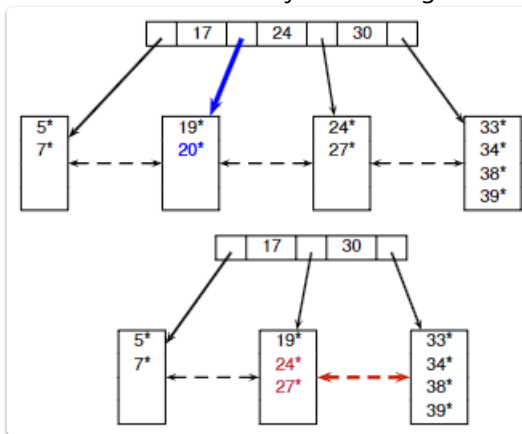
- Avoid splitting by **redistributing** entries when no-full **adjacent** sibling node (no additional sibling searches)



- When deleting, when entries $< d$ check if there are adjacent siblings to redistribute from



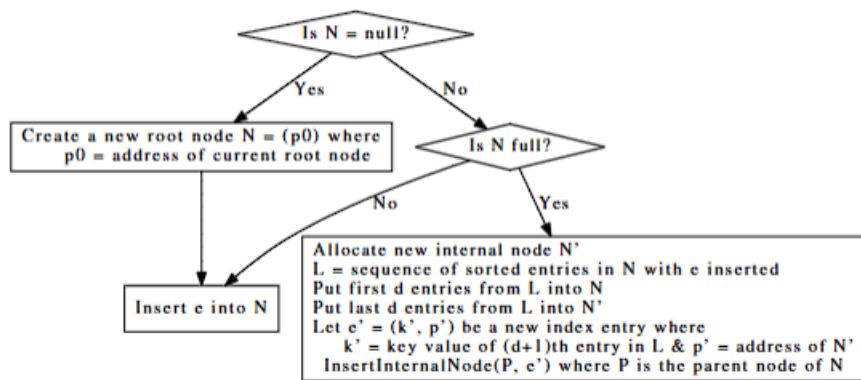
- If underflowed node's adjacent sibling nodes has exactly d entries, need to merge



Algorithms Summarized

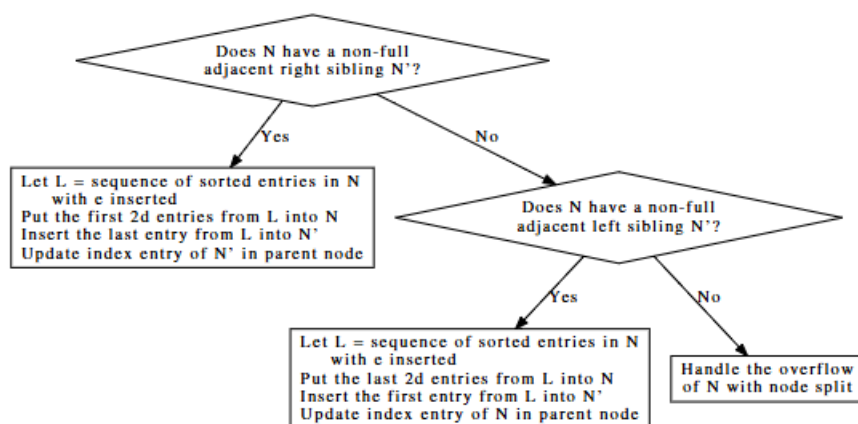
InsertInternalNode(N, e)

Insert index entry e into internal index node N
 d = order of index



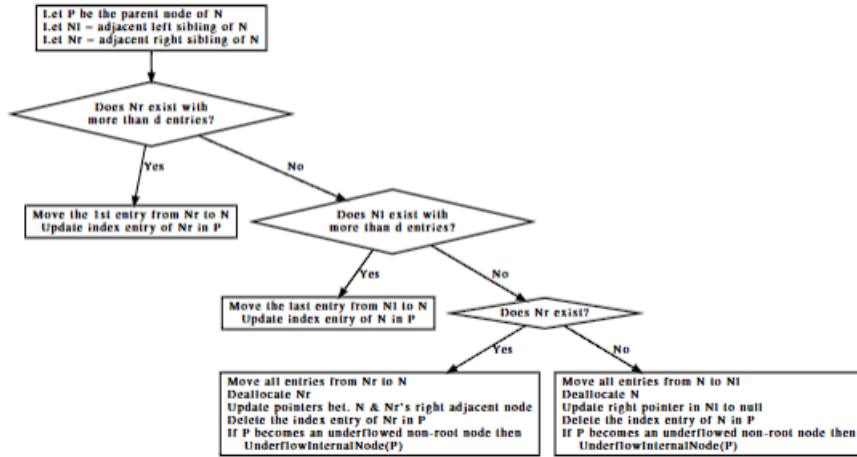
Redistribution of data entries (in leaf nodes)

e = new data entry to be inserted into a full leaf node N
 d = order of index

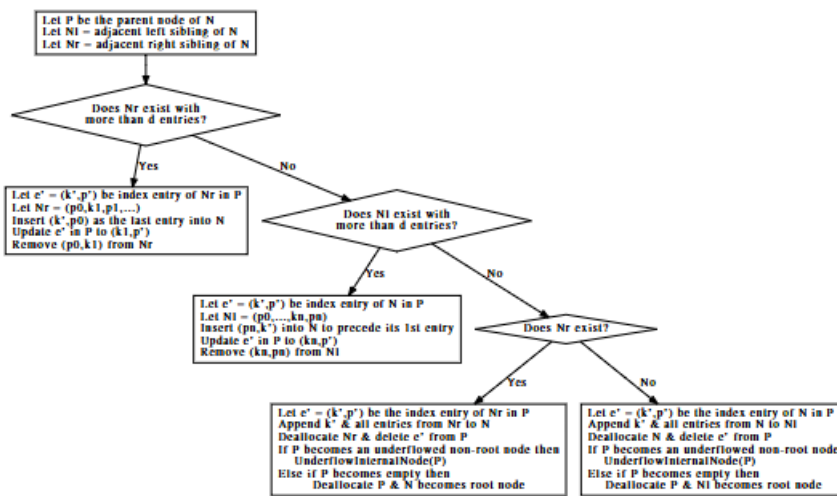


B⁺-tree: Deletion Algorithm

N = non-root leaf node that underflows after deletion of a data entry
Assume redistribution is attempted whenever possible



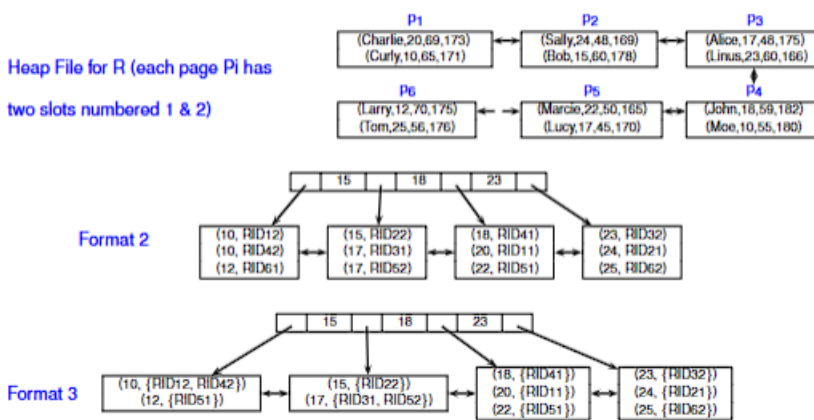
UnderflowInternalNode(N)



Data Formats

- Format 1: k^* is an actual data record (with search key value k)
- Format 2: k^* is of the form (k, rid) , where rid is the record identifier of a data record with search key value k
- Format 3: k^* is of the form $(k, rid-list)$, where $rid-list$ is a list of record identifiers of data records with search key value k

- An index using format 2 or 3 for its data entries is known as an **unclustered index**



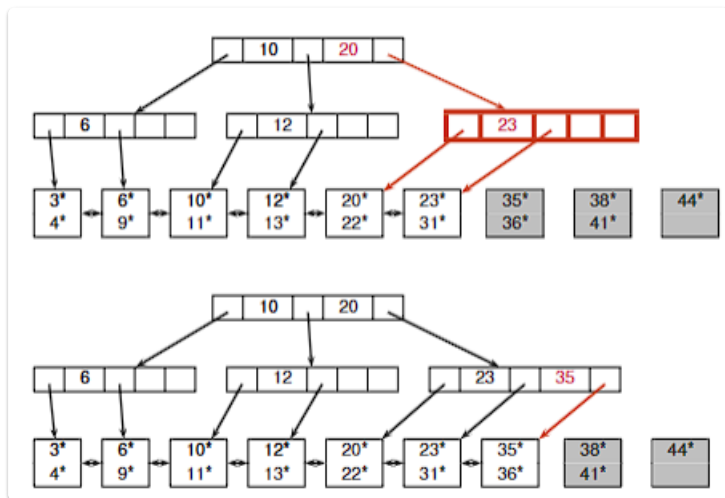
Relations, Indices and B⁺-Tree

- Each relation stored in heap file or clustered index

- Heap File: Each data entry refers to tuple using RID
- Clustered Index (B^+ -Tree): Data entry refers to tuple using clustering key value, and need to be searched to retrieved matching data
- At most 1 clustered index for each relation

Bulk-Loading

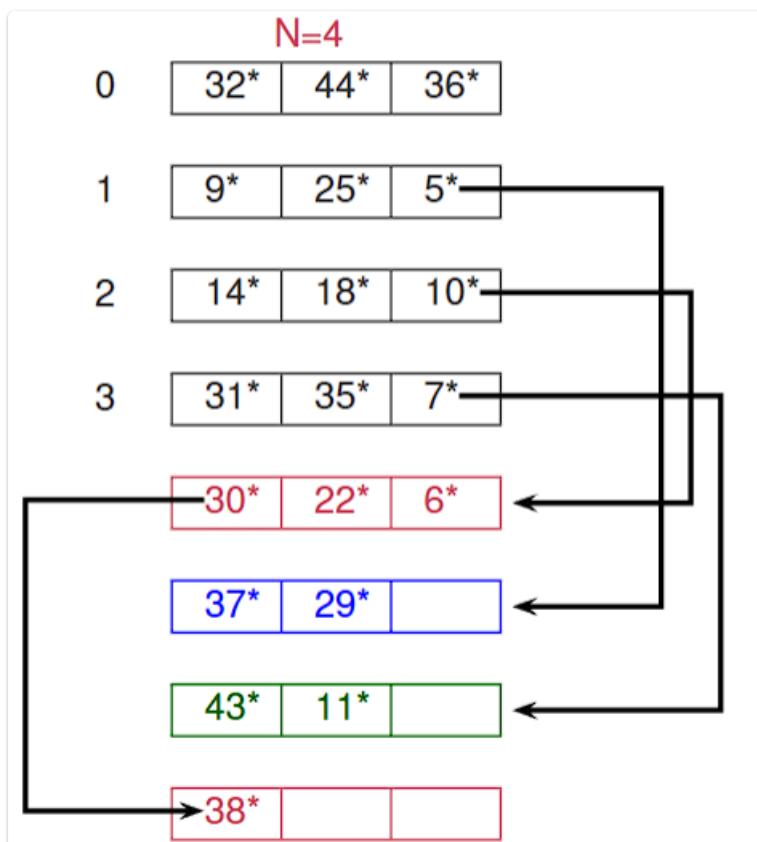
1. Sort data by search key
2. Load leaf pages into tree with sorted entries
3. Initialize B^+ -Tree with empty root page
4. For each leaf, insert index entry into rightmost parent-of-leaf level page of B^+ -Tree



3. Hash-Index

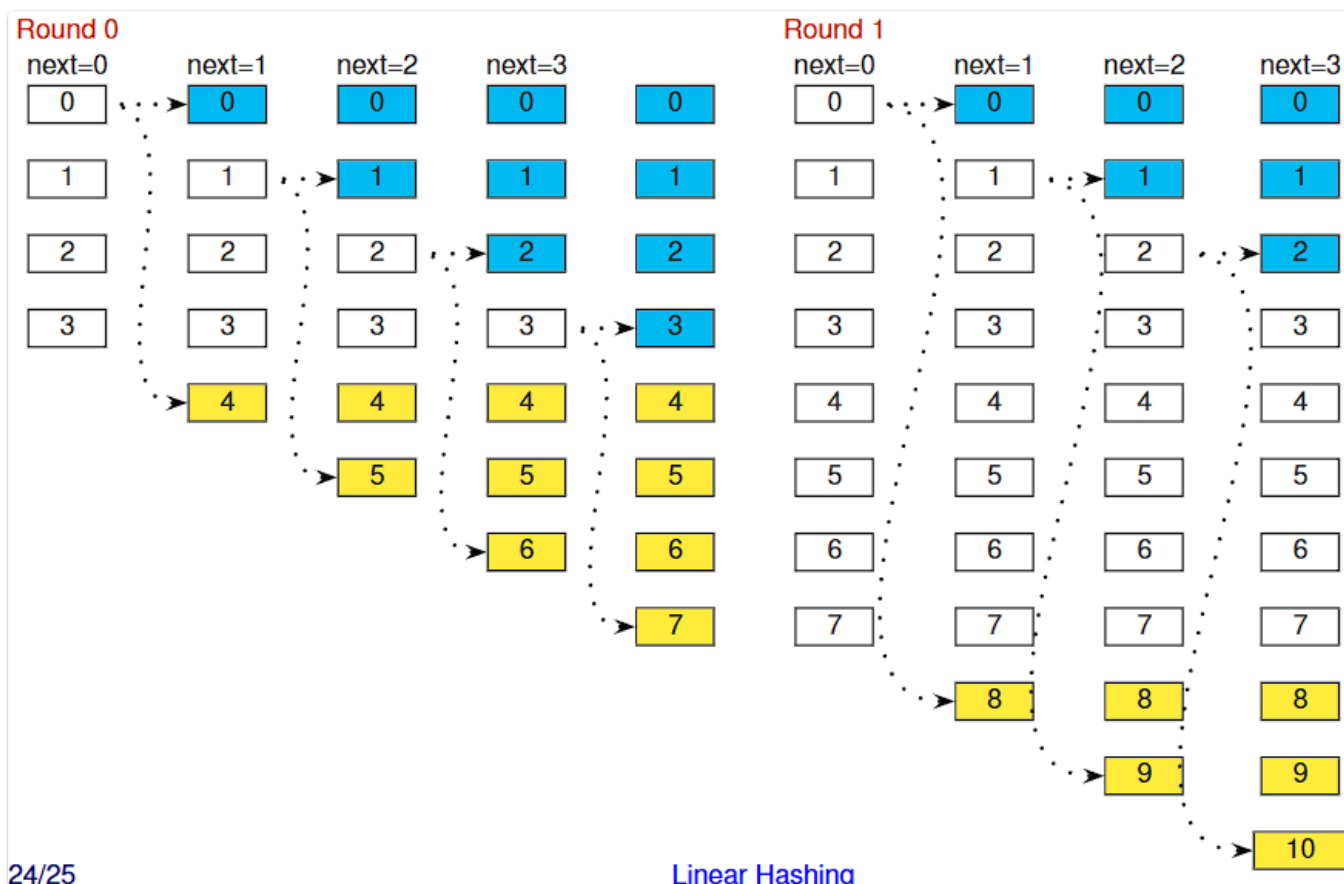
Static Hashing

- Hash function $h(\cdot)$ gives you bucket an inserted to a bucket B_i where $i = h(k) \bmod N$
- Each bucket has a primary page (with multiple records) and a chain of zero or more **overflow pages** (LinkedList)



Linear Hashing

- Dynamic (Size of hash table grows)
- Hash file grows linearly by systematic splitting of buckets
- **Overflow pages still needed** since overflowed bucket may not split immediately
- For each **round** of splitting, all buckets should be split
 - Total number of buckets in round $i = N_i = 2^i N_0$
- Implementation details
 - Bucket no. of split image is continuous
 - e.g. {0,1,2,3} will become {0...7} with {0,4}, {1,5} as split image
 - $B_j = B_{N_i+j}$
 - Maintain a variable **next** to specify next bucket to be split
 - Initialized to 0 at the start of each round, incremented each split



24/25

Linear Hashing

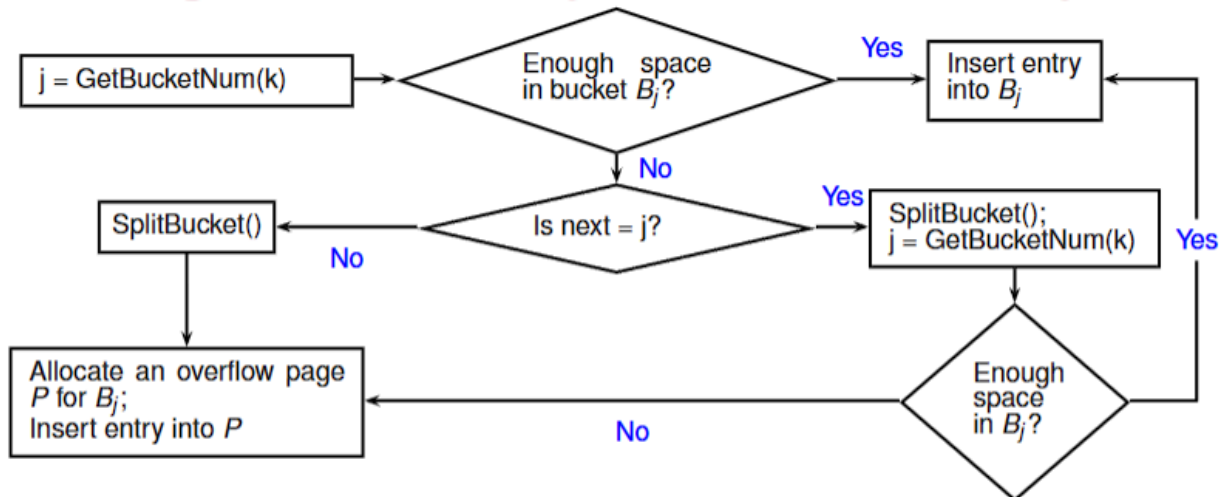
Splitting

- When?
 - When overflows (Assume this for course)
 - When space utilization above threshold
- How?
 - Apply hash value for round N
 - Compare if the bucket has been split using **next** variable
 - If bucket $i < \text{next}$, means already split
 - If split already, apply round $N + 1$'s hash function and modulo for round $N + 1$ or allocate overflow page (See below)
 - For convenience, we can split using powers of 2, then use the MSB to determine whether it's the split image

	Buckets that have been split	Buckets yet to split	Split images
Bucket index	0 ... next - 1	next ... $N_i - 1$	$N_i ... N_i + \text{next} - 1$

	Buckets that have been split	Buckets yet to split	Split images
Hash function	Use $h_i + 1$	Use h_i	Use $h_i + 1$

Inserting a data entry with search key k



- Let **level** denote the current round number
- **GetBucketNum(k)** returns bucket # where entry with search key k is located

$$GetBucketNum(k) = \begin{cases} h_{level}(k) & \text{if } h_{level}(k) \geq next, \\ h_{level+1}(k) & \text{otherwise.} \end{cases}$$

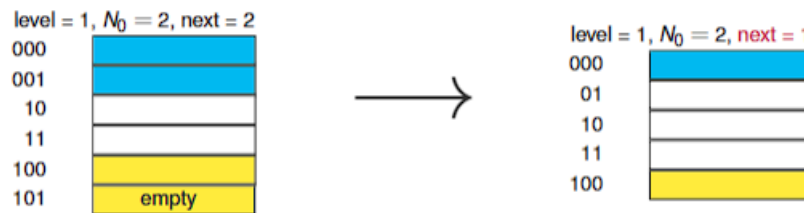
- **SplitBucket()** splits bucket B_{next}
 1. Redistribute the entries in B_{next} between B_{next} & $B_{next+N_{level}}$ using $h_{level+1}()$
 2. $next = next + 1$
 3. if $(next = N_{level})$ then { level = level + 1; next = 0 }

Deletion

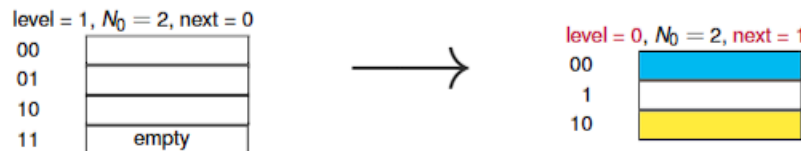
- Only remove when last bucket empty

Linear Hashing: Deletion

- Locate bucket and delete entry. If the last bucket $B_{N_{level}+next-1}$ becomes empty, it can be removed
- **Case 1:** If $next > 0$, then decrement $next$ by one

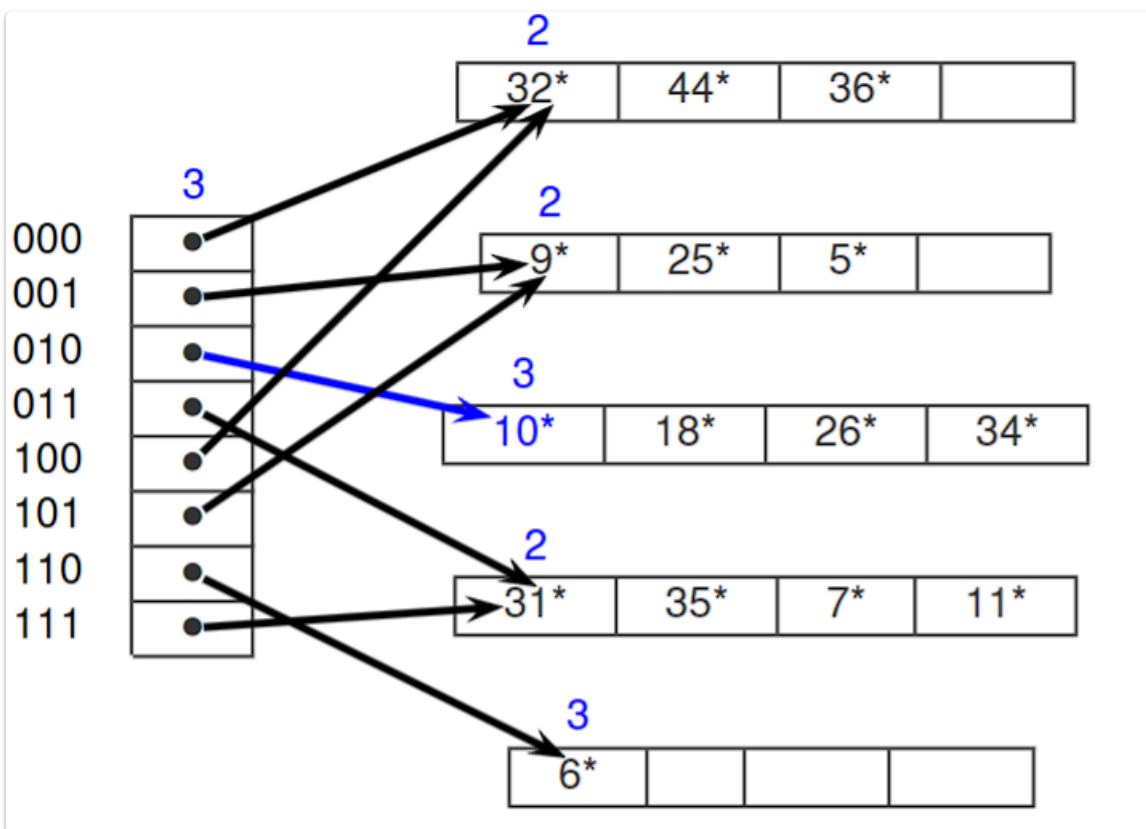


- **Case 2:** $next = 0$ and $level > 0$
 - ▶ Update $next$ to point to the last bucket in previous level $B_{N_{level-1}-1}$
 - ▶ Decrement $level$ by one



Extendible Hashing

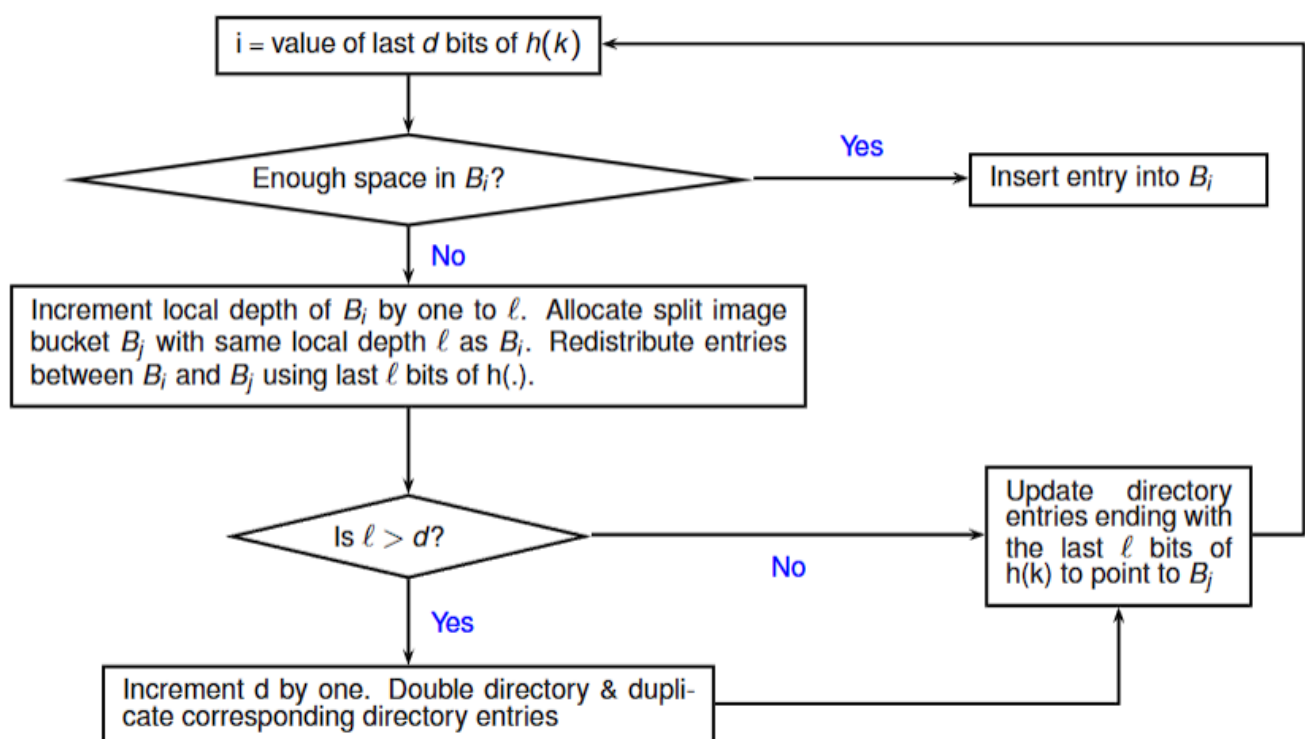
- Similar to Linear Hashing
- No overflow pages
- Solve overflow by **splitting overflowed bucket**
- Overflow page only used when all bits used
 - Number of collisions exceed page capacity
- Use directory of pointers to buckets



Implementation

- Maintain a local depth (Kind of like rounds)
 - If local depth < global depth, it means there's multiple pointers pointing to it
- When splitting
 - If local depth = global depth, we double the size of the directory
 - If local depth < global depth, it means we have a spare pointer

Inserting data entry with search key k



Extendible Hashing: Deletion

- Locate bucket B_i containing entry & delete entry
 - ▶ B_i denote the bucket pointed by the directory entry with index i
- If B_i becomes empty, B_i can be merged with the bucket B_j where both buckets have the same local depth ℓ and i & j differ only in the last ℓ^{th} bit
 - ▶ B_i is deallocated
 - ▶ B_j 's local depth is decremented by one
 - ▶ Directory entries that point to B_i are updated to point to B_j
- More generally, B_i & B_j (with same local depth ℓ and i & j differ only in last ℓ^{th} bit) can be merged if their entries can fit within a bucket
- If each pair of corresponding entries point to the same bucket, directory can be halved
 - ▶ d is decremented by one

4a. Query Evaluation: Selection

Query Processing

1. Query Parser and Translator (into relational algebra expression)
2. Query Optimizer (into query plan)
3. Query Evaluator (into query result)

Selection: $\sigma_p(R)$

Access Path

Methods

- Table scan: Scan all data pages
- Index scan: Scan index pages
- Index combination: Combine results from multiple index scans (e.g. using intersect and joins)

Selectivity of Access Path

- **Number of index & data pages** retrieved to access the data records/entries
- The most selective access path (smallest selectivity) retrieves smallest number of pages

Index-only Plans

- If the plan does not need to access any data tuples
- i.e. if there's an index for it, we only need the index to get the plan

Using Include Columns

```
create index weight_height_index on Student(weight) include (height);
```

```
select height from Student where weight between 51 and 59
```

- The index-only plan for the above query only becomes available when we add an **include column** on `Student.height`

Covering Index

- If all of relation R 's attributes referenced in query Q are part of the key of include columns in the index

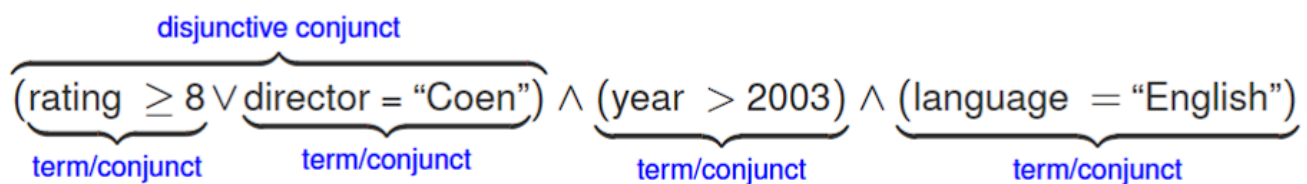
B^+ -Tree Cost Analysis

Conjunctive Normal Form (CNF) Predicates

If a table scan is used, and then union with another indexed column condition, there's a chance we don't need to even use the index

Used to evaluate the cost

- Term: $R.A \text{ op } c$
- Conjunct: One or more terms connected by $\vee$
 - Conjunct is disjunctive if it contains a $\vee$
- CNF: One or more conjuncts connected by $\wedge$



B^+ -Tree Index Scan Cost Analysis

$$\text{Cost of index scan} = N_{\text{internal}} + N_{\text{leaf}} + N_{\text{lookup}}$$

Notation	Meaning
r	relational algebra expression
$ r $	number of tuples in output of r
$ r $	number of pages to store the output of r
b_d	number of data records that can fit on a page
b_i	number of data entries that can fit on a page
F	average fanout of B^+ -tree index (i.e., number of pointers to child nodes)

N_{internal}

- To get number of internal nodes accessed, we simply need to see height of B^+ -Tree
- Assumption: scan 1 path in the internal node

$$h = \begin{cases} \lceil \log_F(\lceil \frac{||R||}{b_d} \rceil) \rceil & \text{if index is a clustered index} \\ \lceil \log_F(\lceil \frac{||R||}{b_i} \rceil) \rceil & \text{otherwise} \end{cases}$$

N_{leaf}

Covered conjuncts

- When all attributes in a conjunct C appear in key K

- When the included columns in an index are all in the predicate

Predicate Matching

An index I matches a predicate p when:

4. Attributes in p form a prefix of composite key K
5. All comparison operators in p must be equality except for the prefix's last attribute

Primary Conjuncts

- Subset of Covered Conjuncts in p that I matches
- Primary conjuncts $\subseteq$ Covered conjuncts

Example:

- B⁺-tree index I with key = (age, weight, height)
- Predicate $p = (\text{age} \geq 18) \wedge (\text{age} \leq 20) \wedge (\text{weight} = 65) \wedge (\text{level} = 3)$
- Primary conjuncts: $(\text{age} \geq 18) \wedge (\text{age} \leq 20)$
- Non-primary conjuncts: $(\text{weight} = 65) \wedge (\text{level} = 3)$

Let p' be the primary conjuncts of p and estimate the number of leaf pages scanned using the primary conjuncts as an upper bound

$$\begin{cases} \lceil \frac{||\sigma_{p'}(R)||}{b_d} \rceil \leq N_{leaf} \leq \lceil \frac{||\sigma_{p'}(R)||}{b_d} \rceil & \text{if index is a clustered index} \\ \lceil \frac{||\sigma_{p'}(R)||}{b_i} \rceil \leq N_{leaf} \leq \lceil \frac{||\sigma_{p'}(R)||}{b_i} \rceil & \text{otherwise} \end{cases}$$

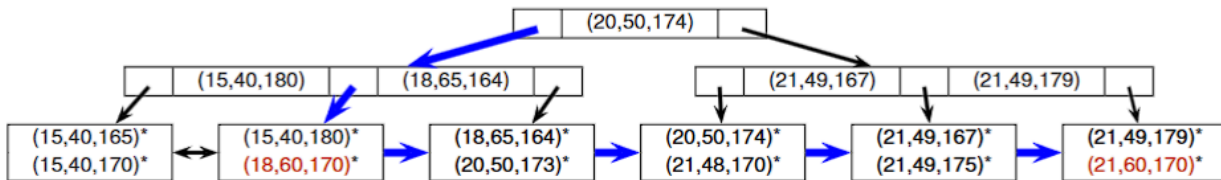
N_{lookup}

- Number of data record to lookup via RID lookups
- If we have a covering index, it would be $N_{lookup} = 0$
- Otherwise it's a range
 - Lower bound is when all records can fit into respective pages fully
 - Upper bound will be when each record is on a different page
 - $\lceil \frac{||\sigma_{p'}(R)||}{b_d} \rceil \leq N_{lookup} \leq \min\{||\sigma_{p'}(R)||, |R|\}$

Worked Example

Example

- Consider a B^+ -tree index I with key = (age,weight,height) on R where R is stored in a heap file
- Query: **select * from R where (age $\geq$ 18) and (weight = 60) and (level = 3)**
 - Covered conjuncts of p : $p_c = (\text{age} \geq 18) \wedge (\text{weight} = 60)$
 - Primary conjuncts of p : $p' = (\text{age} \geq 18)$
- Assume the following:
 - $|R| = 6$, $||R|| = 12$, $||\sigma_{p'}(R)|| = 9$, $||\sigma_{p_c}(R)|| = 2$
 - $b_d = b_i = 2$, Height of $I = 2$
- Cost of index scan with $I = N_{\text{internal}} + N_{\text{leaf}} + N_{\text{lookup}} = 2 + 5 + 2 = 9$
 - $1 = \lceil \frac{2}{2} \rceil = \lceil \frac{||\sigma_{p_c}(R)||}{b_i} \rceil \leq N_{\text{leaf}} \leq \lceil \frac{||\sigma_{p'}(R)||}{b_i} \rceil = \lceil \frac{9}{2} \rceil = 5$
 - $1 = \lceil \frac{2}{2} \rceil = \lceil \frac{||\sigma_{p_c}(R)||}{b_d} \rceil \leq N_{\text{lookup}} \leq \min\{||\sigma_{p_c}(R)||, |R|\} = \min\{2, 6\} = 2$



Hash-Index Cost Analysis

$$\text{Cost of hash index scan} = N_{\text{dir}} + N_{\text{bucket}} + N_{\text{lookup}}$$

Assuming R is stored in a heap file

- N_{dir}
 - Number of index's directory pages accessed
 - 1 if extendible hash index, else 0
- N_{bucket}
 - Number of index's primary/overflow pages accessed
 - M_{bucket} = Maximum number of primary/overflow pages for a bucket
 - $\lceil \frac{||\sigma_{p'}(R)||}{b_d} \rceil \leq N_{\text{bucket}} \leq M_{\text{bucket}}$
- N_{lookup}
 - Number of pages accessed to retrieve records
 - Refer to B^+ -Tree for formula

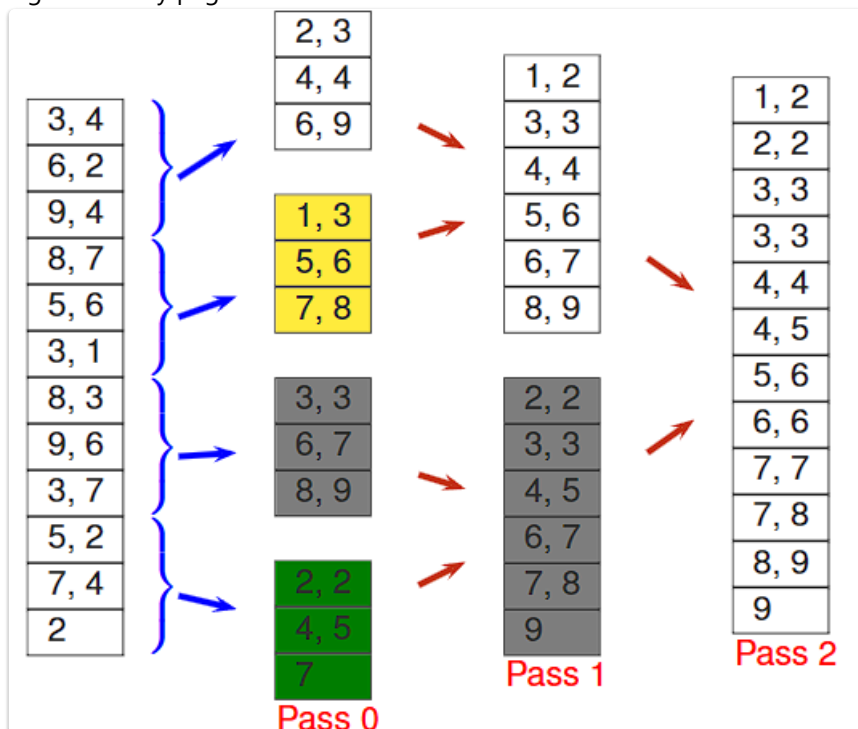
4b. Query Evaluation: Sorting

External Merge Sort

Overview

- Each one creates a new sorted temporary file using a quicksort
- Merges files by reading 1st page entry of each temp file and increment pointers accordingly

- e.g. 3 memory pages



Analysis

- File size of N pages
- Number of memory pages available for sorting = B
- Pass 0:** Creation of sorted runs
 - Read in and sort B pages at a time
 - Number of sorted runs created = $\lceil N/B \rceil$
 - Size of each sorted run = B pages (except possibly for last run)
- Pass $i, i \geq 1$:** Merging of sorted runs
 - Use $B - 1$ memory pages for input & one memory page for output
 - Performs $(B-1)$ -way merge
- Analysis:**
 - N_0 = number of sorted runs created in pass 0 = $\lceil N/B \rceil$
 - Total number of passes = $\lceil \log_{B-1}(N_0) \rceil + 1$
 - Total number of I/O = $2N(\lceil \log_{B-1}(N_0) \rceil + 1)$
 - ★ Each pass reads N pages & writes N pages

Sorting using B^+ -Tree

- If the key for sorting K_{sort} is a prefix of K_{index} , query can be evaluated without sorting
- If clustered index, sequentially scan index's leaf pages to retrieve data records in sorted order
 - Otherwise, sequentially scan index's leaf pages and for each scanned leaf page, retrieve data records via RID lookup

5. Query Evaluation: Projection & Join

$\pi_{A_1, A_2}(R)$ means projections or "take attributes A_1, A_2 "

π only takes distinct

π^* includes duplicates

Operation

Involves 2 tasks

1. Remove unwanted attributes
2. Eliminate duplicate tuples (as required)

2 Approaches: Sorting and hashing

Sort-Based Approach

1. Extract attributes from records
2. Sort records using attributes as sort key
3. Remove duplicates using sequential scan

Cost Analysis

Consider $\pi_L(R)$ where L denote some sequence of attributes of R

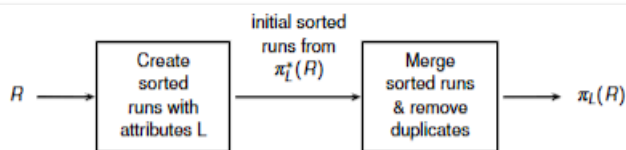


Cost Analysis: Assume B number of memory pages allocated for $\pi_L(R)$

- **Step 1:**
 - ▶ Cost to create $\pi_L^*(R)$ from $R = |R| + |\pi_L^*(R)|$
- **Step 2:**
 - ▶ N_0 = number of initial sorted runs = $\lceil |\pi_L^*(R)| / B \rceil$
 - ▶ Number of merging passes = $\lceil \log_{B-1}(N_0) \rceil$
 - ▶ Cost to sort records = $2|\pi_L^*(R)| (\lceil \log_{B-1}(N_0) \rceil + 1)$
- **Step 3:**
 - ▶ Cost to scan sorted records to remove duplicates = $|\pi_L^*(R)|$

Optimized Sort-Based Approach

- Split sorting into created sorted runs, and merging
- Then extract + sort at same time
- And merge + remove duplicates at same time

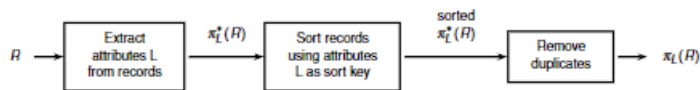


Cost Analysis: Assume B number of memory pages allocated for $\pi_L(R)$

- **Step 1:**
 - ▶ Size of each initial sorted run = $B - 1$ pages
 - ▶ N_0 = number of initial sorted runs = $\lceil |\pi_L^*(R)| / (B - 1) \rceil$
 - ▶ Cost to create initial sorted runs = $|R| + |\pi_L^*(R)|$
- **Step 2:**
 - ▶ Number of merging passes = $\lceil \log_{B-1}(N_0) \rceil$
 - ▶ Cost of merging passes = $2|\pi_L^*(R)| \lceil \log_{B-1}(N_0) \rceil$
 - ▶ Cost of merging passes excluding I/O cost to output $\pi_L(R) = (2\lceil \log_{B-1}(N_0) \rceil - 1)|\pi_L^*(R)|$

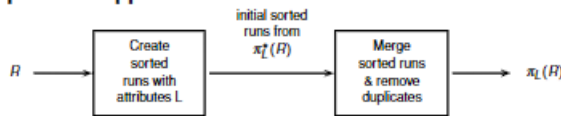
Comparison

- **Cost of non-optimized approach**



$$|R| + 2|\pi_L^*(R)| (\lceil \log_{B-1}(N_0) \rceil + 2) \text{ where } N_0 = \lceil |\pi_L^*(R)| / B \rceil$$

- **Cost of optimized approach**

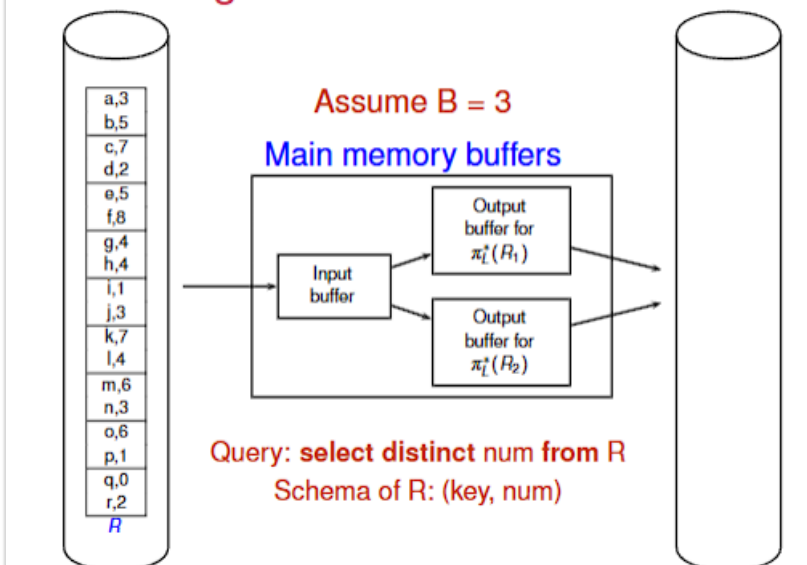


$$|R| + 2|\pi_L^*(R)| \lceil \log_{B-1}(N_0^{opt}) \rceil \text{ where } N_0^{opt} = \lceil |\pi_L^*(R)| / (B-1) \rceil$$

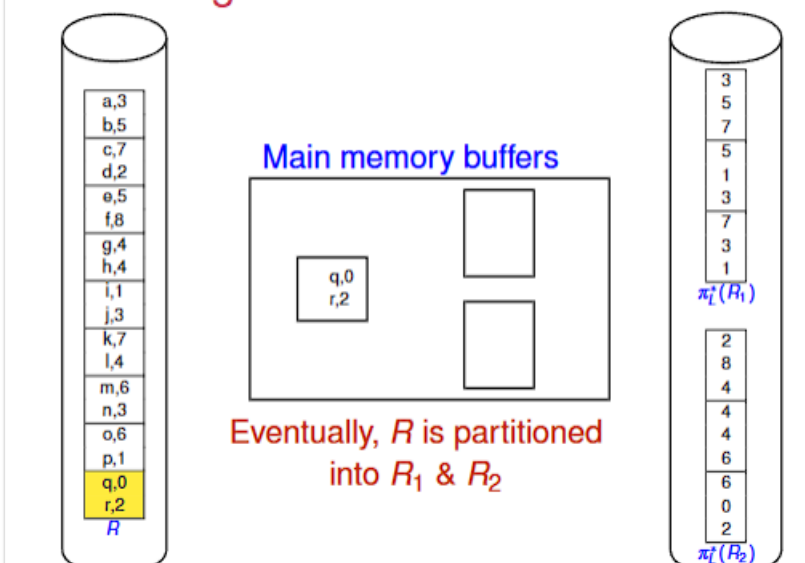
Hash-Based Approach

- Output hash table after insertion of values (without inserting if already included)
- If results cannot fit in main memory, we need to partition
- Partition by hashing the chosen attributes L , then the duplicates will go to the same bucket which can be safely removed

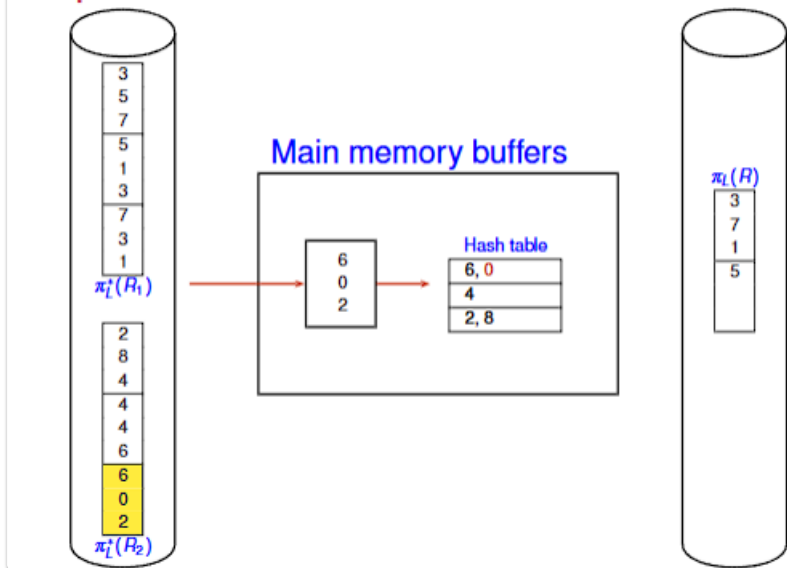
Partitioning Phase



Partitioning Phase



Duplicate Elimination Phase



Partition Overflow

- When we create the initial partition by hash value, it may still not be enough to fit in the main memory for the hash table in duplicate elimination phase
- Re-partition into smaller buckets before doing duplicate elimination

Hash-based Approach: Analysis

- Approach is effective if the number of allocated memory pages B is large relative to $|R|$
- How large should B be?
 - ▶ Assume that h distributes tuples in R uniformly
 - ▶ Each R_i has $\frac{|\pi_L^*(R)|}{B-1}$ pages
 - ▶ Size of hash table for each $R_i = \frac{|\pi_L^*(R)|}{B-1} \times f$
 - ★ $f = \text{fudge factor}, f > 1$
 - ▶ Therefore, to avoid partition overflow, $B > \frac{|\pi_L^*(R)|}{B-1} \times f$
 - ★ Approximately, $B > \sqrt{f \times |\pi_L^*(R)|}$
- Analysis: Assume there's no partition overflow
 - ▶ Cost of partitioning phase: $|R| + |\pi_L^*(R)|$
 - ▶ Cost of duplicate elimination phase: $|\pi_L^*(R)|$
 - ▶ Total I/O cost = $|R| + 2|\pi_L^*(R)|$

Joins

- In Relation Operator $R \bowtie S$, R is the outer relation and S is the inner relation

Iteration-Based (Block Nested Loop)

Naive way:

```

for each tuple in R
  for each tuple in S
    if r matches s, add output (r,s) to result
    
```

Considering pages:

```

for each page  $R_i$  of  $R$ 
  for each tuple  $r \in R_i$ 
    for each page  $S_j$  of  $S$ 
      if  $r$  matches  $s$ , add output (r,s) to result
    
```

for each tuple $s \in$
if r matches s , add output (r,s) to result

IO Cost Analysis: $|R| + ||R||$

$|R|$: Times to scan table R

$||R||$: No. of tuples in R

Page-based nested loop join

Reduces no. of times we scan R

for each page R of R
for each page of
for each tuple $r \in R$
for each tuple $s \in$
if r matches s , add output (r,s) to result

IO Cost Analysis: $|R| + |R|$

$|R|$: Times to scan table R

Block Based

- Allocate 1 page for output, 1 page for , remaining for R
- Reads R in chunks of $B - 2$ pages

Index-Based (Index Nested Loop)

- Change inner table scan with index scan
- For all tuples in R do a index scan for

Partition-Based

Sort-Merge Join

- Sort both tables, then do similar to external sort merging, find similar matching joins
- Run merge during sort and merge to join at the same time
- Merge until $B > N(R, i) + N(, j)$, where i and j is the stage

Hash Join

- Grace Hash Join
- Use inner relation to build hash table (build relation)
- Use outer relation to query (probe relation)

Steps:

- Partition Phase: Partition based on hash algorithm so it can fit into a hash table
- Probing Phase
 - Build hash table to compute join or $(R_{1,1})...$, allocating 1 buffer for input and 1 for output
 - When hash table full, save to disk

When to use each algorithm (Complex cases)

- Multiple equality-join: Index-nested loop join, Sort-merge, other algorithms changed
- Inequality-join: Index-nested loop join (Sort-merge and Hash-based too difficult)